

Vertex Developer Guide

Version: 1.0

Status: Draft

1. Objetivo

Este guia apresenta o fluxo oficial de desenvolvimento para a plataforma Vertex.

Seu objetivo é orientar desenvolvedores na criação, teste, distribuição e manutenção de plugins seguindo os padrões da plataforma.

Este documento complementa a Plugin Specification, focando no processo de desenvolvimento em vez da arquitetura interna.

2. Pré-requisitos

Antes de desenvolver um plugin, recomenda-se conhecimento em:

- Kotlin;
- Android;
- Coroutines;
- Jetpack Compose (quando aplicável);
- Git;
- Arquitetura do Vertex.

Também é recomendada a leitura dos seguintes documentos:

- Vision & Philosophy
- System Architecture
- Core Specification
- Plugin Specification

3. Criando um Plugin

Todo plugin segue o mesmo fluxo de desenvolvimento.

Etapas

Definir o objetivo do plugin.

Exemplos:

- Launcher
- Watchface
- Player de música
- Cliente de mensagens
- Navegação
- Monitor cardíaco

Etapa 2

Criar o Manifest.

O Manifest define:

- ID
- Nome
- Versão
- Permissões
- Capabilities
- Eventos
- Estados
- Dependências

Sem um Manifest válido o plugin não pode ser instalado.

Etapa 3

Implementar a lógica.

A lógica deve utilizar exclusivamente:

- Plugin API
- PluginContext

Nunca APIs internas do Core.

Etapa 4

Criar a interface.

A interface deve seguir os padrões da UI do Vertex.

Ela deve ser otimizada para telas redondas e diferentes tamanhos de smartwatch.

Etapa 5

Executar testes.

Todo plugin deve ser validado antes da publicação.

4. Organização do Projeto

Estrutura recomendada:

```
MyPlugin/  
├── manifest/  
│   └── plugin.json  
├── src/  
│   ├── main/  
│   ├── ui/  
│   ├── capabilities/  
│   ├── events/  
│   ├── state/  
│   ├── services/  
│   └── storage/  
├── assets/  
├── resources/  
├── tests/  
├── build.gradle.kts  
└── README.md
```

Essa organização facilita manutenção e escalabilidade.

5. Arquitetura Recomendada

Todo plugin deve ser dividido em camadas.

Exemplo:

- UI
- Domain
- Services
- Storage
- Communication

Evite misturar responsabilidades.

6. Comunicação

Plugins devem utilizar apenas:

- Capabilities
- Events
- State
- Shared Objects

Nunca acessar diretamente outro plugin.

7. Gerenciamento de Estado

Sempre utilizar o State Engine.

Evitar variáveis globais e estados duplicados.

Estados devem ser pequenos, previsíveis e facilmente sincronizáveis.

8. Interface

A interface deve:

- funcionar em telas redondas;
- adaptar-se a diferentes resoluções;
- minimizar consumo de bateria;
- evitar animações desnecessárias.

Sempre utilizar os componentes oficiais do Vertex quando disponíveis.

9. Performance

Plugins devem:

- evitar trabalho na thread principal;
- carregar recursos sob demanda;
- reutilizar objetos sempre que possível;

- minimizar alocações frequentes;
- liberar recursos não utilizados.

Todo plugin deve respeitar o Budget definido pelo Core.

10. Segurança

Plugins devem solicitar apenas as permissões realmente necessárias.

Nunca armazenar informações sensíveis sem utilizar os mecanismos oficiais da plataforma.

Todo acesso ao Android deve ocorrer através da Plugin API.

11. Logging

Utilizar exclusivamente o Logging Framework do Vertex.

Logs devem ser:

- claros;
- úteis;
- objetivos.

Evitar registrar informações sensíveis.

12. Tratamento de Erros

Todo erro esperado deve ser tratado.

Plugins nunca devem encerrar abruptamente.

Sempre que possível:

- recuperar;
- registrar;
- continuar funcionando.

13. Testes

Todo plugin deve possuir testes automatizados.

Recomenda-se:

Testes Unitários

Validam classes e funções isoladas.

Testes de Integração

Validam comunicação com:

- State
- Events
- Capabilities
- Storage

Testes de Interface

Validam:

- layout;
- navegação;
- responsividade;
- interação.

Testes de Performance

Avaliam:

- CPU;
- memória;
- bateria;
- tempo de inicialização.

Testes de Compatibilidade

Verificam funcionamento em diferentes versões do Core.

14. Build

Todo plugin deve utilizar o Build System oficial do Vertex.

Durante o build são executadas verificações como:

- Manifest;
- dependências;
- assinatura;
- compatibilidade;
- lint;
- testes;
- empacotamento.

Builds que não atendam aos requisitos podem ser rejeitados.

15. Publicação

Fluxo recomendado:

1. Build.
2. Testes.
3. Assinatura.
4. Validação.
5. Publicação.

Após publicado, o plugin pode ser distribuído através da Plugin Store ou por instalação manual, conforme as políticas da plataforma.

16. Atualizações

Novas versões devem:

- manter compatibilidade quando possível;
- atualizar o Manifest;
- documentar mudanças;
- preservar dados do usuário sempre que possível.

17. Versionamento

Todo plugin deve seguir versionamento semântico.

Formato:

MAJOR.MINOR.PATCH

- MAJOR: mudanças incompatíveis.
- MINOR: novas funcionalidades compatíveis.
- PATCH: correções e melhorias.

18. Boas Práticas

- Mantenha responsabilidade única por plugin.
- Utilize apenas APIs públicas.
- Evite código duplicado.
- Escreva testes desde o início.
- Documente APIs e comportamentos importantes.
- Minimize consumo de recursos.
- Trate exceções adequadamente.
- Prefira simplicidade à complexidade.
- Projete pensando na evolução futura.

19. Checklist antes da Publicação

- Manifest válido.
- Permissões revisadas.
- Testes aprovados.
- Sem erros críticos de lint.
- Compatibilidade verificada.
- Assinatura válida.
- Documentação atualizada.
- Performance dentro dos limites.
- Consumo de memória aceitável.
- Interface adaptada para telas redondas.

20. Resumo

O desenvolvimento de plugins no Vertex é baseado em um conjunto de padrões que priorizam modularidade, desempenho, segurança e facilidade de manutenção.

Seguir este guia garante que plugins sejam consistentes entre si, utilizem corretamente os serviços da plataforma e proporcionem uma experiência previsível tanto para usuários quanto para outros desenvolvedores.